

React Interview Questions

Comprehensive Guide with Code Examples

30 Questions

8 Topics

With Code Examples

Beginner → Advanced

Table of Contents

Core Concepts	Q1–Q5
Hooks	Q6–Q13
Performance Optimization	Q14–Q16
State Management	Q17–Q18
Advanced Patterns	Q19–Q23
Lifecycle & Reconciliation	Q24–Q26
Routing & Forms	Q27–Q28
Testing & TypeScript	Q29–Q30

■ Core Concepts

Q1. What is React and what are its key features?

React is a JavaScript library for building user interfaces developed by Meta. Key features: (1) Virtual DOM for efficient updates, (2) Component-based architecture, (3) Unidirectional data flow, (4) JSX syntax, (5) Declarative UI.

Q2. What is JSX? Why do we use it?

JSX (JavaScript XML) is a syntax extension that lets you write HTML-like code inside JavaScript. It improves readability and is compiled by Babel into `React.createElement()` calls.

```
const el = <h1 className="title">Hello, {name}!</h1>;
// compiles to:
const el = React.createElement('h1', {className:'title'}, `Hello, ${name}!`);
```

Q3. What is the Virtual DOM and how does React use it?

The Virtual DOM is a lightweight in-memory representation of the real DOM. When state changes, React creates a new Virtual DOM tree, diffs it against the previous one (reconciliation), and updates only the changed real DOM nodes — minimizing expensive repaints.

■ *State change → new VDOM → diff (reconciliation) → minimal real DOM patch*

Q4. What are React Components? Class vs Functional?

Components are reusable UI building blocks. Class components use ES6 classes and lifecycle methods. Functional components are plain functions — simpler, preferred with Hooks.

```
// Functional (preferred)
function Greet({ name }) {
  return <h2>Hello, {name}</h2>;
}

// Class
class Greet extends React.Component {
  render() { return <h2>Hello, {this.props.name}</h2>; }
}
```

Q5. What are Props? How are they different from State?

Props are read-only inputs passed from parent to child. State is mutable data managed inside the component. Props flow down; state changes trigger re-renders.

```
function Card({ title, count }) { // props
  const [likes, setLikes] = useState(0); // state
  return <div>{title} – Likes: {likes}</div>;
}
```

■ Hooks

Q6. What are React Hooks? Why were they introduced?

Hooks (React 16.8+) let functional components use state, lifecycle, and other React features without classes. They promote reuse via custom hooks and eliminate confusing 'this' binding.

Q7. Explain useState Hook with example.

useState returns [currentValue, setter]. Calling the setter schedules a re-render with the new value.

```
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>+1</button>
    </>
  );
}
```

■ Always use functional update (prev => prev+1) when new state depends on old state.

Q8. Explain useEffect Hook — when does it run?

useEffect runs side-effects after render. Dependency array controls when: [] = mount only, [dep] = when dep changes, omitted = every render. Return a cleanup function to avoid memory leaks.

```
useEffect(() => {
  const id = setInterval(() => setTick(t => t+1), 1000);
  return () => clearInterval(id); // cleanup
}, []); // [] = run once on mount
```

Q9. What is useRef? Give practical examples.

useRef returns a mutable ref object whose .current property persists across renders without triggering re-renders. Uses: DOM access, storing timers, previous values.

```
function Focus() {
  const inputRef = useRef(null);
  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current.focus()}>Focus</button>
    </>
  );
}
```

Q10. Explain useMemo and useCallback — what problems do they solve?

useMemo memoizes an expensive computed value; recomputes only when deps change. useCallback memoizes a function reference; prevents unnecessary re-renders of children.

```
const sorted = useMemo(
  () => [...items].sort((a,b) => a-b),
  [items] // only re-sort when items changes
);

const handleClick = useCallback(() => {
  doSomething(id);
}, [id]);
```

Q11. What is useContext and when would you use it?

useContext lets any component consume a context value without prop-drilling. Best for: theme, locale, auth state — data needed across many levels.

```
const ThemeCtx = createContext('light');

function App() {
  return <ThemeCtx.Provider value="dark"><Page /></ThemeCtx.Provider>;
}

function Button() {
  const theme = useContext(ThemeCtx);
  return <button className={theme}>Click</button>;
}
```

Q12. What is useReducer? When to prefer it over useState?

useReducer is like useState but for complex state logic. Prefer it when: next state depends on previous, multiple sub-values, or actions resemble Redux patterns.

```
const reducer = (state, action) => {
  switch (action.type) {
    case 'inc': return { count: state.count + 1 };
    case 'dec': return { count: state.count - 1 };
    default: return state;
  }
};

const [state, dispatch] = useReducer(reducer, { count: 0 });
dispatch({ type: 'inc' });
```

Q13. What are Custom Hooks? How do you create one?

Custom hooks are functions starting with 'use' that encapsulate reusable stateful logic. They can call other hooks.

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    fetch(url).then(r=>r.json()).then(d=>{
      setData(d); setLoading(false);
    });
  }, [url]);
  return { data, loading };
}
```

■ Performance Optimization

Q14. What is React.memo? How does it help performance?

React.memo is a HOC that shallow-compares props. If props didn't change, the component skips re-rendering. Use for pure functional components with expensive renders.

```
const ExpensiveList = React.memo(function List({ items }) {
  return <ul>{items.map(i => <li key={i.id}>{i.name}</li>)}</ul>;
});

// Custom comparison:
React.memo(List, (prev, next) => prev.id === next.id);
```

■ *React.memo only does shallow comparison — for deep objects, provide a custom comparator.*

Q15. What is code splitting and lazy loading in React?

Code splitting breaks the bundle into chunks loaded on demand. React.lazy + Suspense enable component-level lazy loading.

```
const Dashboard = React.lazy(() => import('./Dashboard'));

function App() {
  return (
    <Suspense fallback=<Spinner />>
      <Dashboard />
    </Suspense>
  );
}
```

Q16. What are keys in lists and why are they important?

Keys help React identify which items changed, were added, or removed during reconciliation. They must be stable and unique among siblings. Avoid using array index as key for dynamic lists.

```
// Good
items.map(item => <li key={item.id}>{item.name}</li>)

// Bad - unstable index key
items.map((item, i) => <li key={i}>{item.name}</li>)
```

■ State Management

Q17. What is prop drilling and how do you avoid it?

Prop drilling is passing data through many intermediate components that don't need it. Solutions: Context API, state management libraries (Redux, Zustand), or component composition.

```
// Context solution
const UserCtx = createContext();

// Top level
<UserCtx.Provider value={user}>
  <App />
</UserCtx.Provider>

// Deep child
const user = useContext(UserCtx);
```

Q18. Explain Redux and its core principles.

Redux is a predictable state container. Core principles: (1) Single source of truth (one store), (2) State is read-only (change via actions), (3) Changes made with pure reducer functions.

```
// Reducer
function counterReducer(state = 0, action) {
  if (action.type === 'INCREMENT') return state + 1;
  return state;
}

// Modern Redux Toolkit slice
const slice = createSlice({
  name: 'counter',
  initialState: 0,
  reducers: { increment: s => s + 1 }
});
```

■ Advanced Patterns

Q19. What are Higher-Order Components (HOCs)?

A HOC is a function that takes a component and returns an enhanced component. It's a pattern for reusing component logic (auth guards, logging, theming).

```
function withAuth(WrappedComponent) {
  return function AuthComponent(props) {
    const { isLoggedIn } = useAuth();
    if (!isLoggedIn) return <Redirect to="/login" />;
    return <WrappedComponent {...props} />;
  };
}

export default withAuth(Dashboard);
```

Q20. What is the Render Props pattern?

Render props share code between components by passing a function as a prop that determines what to render.

```
function MouseTracker({ render }) {
  const [pos, setPos] = useState({x:0, y:0});
  return (
    <div onMouseMove={e => setPos({x:e.clientX, y:e.clientY})}>
      {render(pos)}
    </div>
  );
}

// Usage:
<MouseTracker render={({x,y}) => <p>{x}, {y}</p>} />
```

Q21. What are Error Boundaries?

Error Boundaries are class components that catch JavaScript errors in their child tree, log them, and display a fallback UI instead of crashing the whole app.

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(err, info) { logError(err, info); }
  render() {
    if (this.state.hasError) return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}
```

■ *Error Boundaries only catch render-phase errors, not event handlers or async code.*

Q22. What are React Portals?

Portals render a child component into a DOM node outside the parent component's hierarchy, while still maintaining React's event bubbling. Useful for modals, tooltips, and dropdowns.

```
function Modal({ children }) {
  return ReactDOM.createPortal(
    <div className="modal">{children}</div>,
    document.getElementById('modal-root') // outside #root
  );
}
```

Q23. What is React Suspense? What problems does it solve?

Suspense lets components 'wait' for something (data, code) before rendering. It shows a fallback UI during loading, enabling declarative loading states.

```
// With React Query / use() hook (React 19+)
function UserProfile({ userId }) {
  const user = use(fetchUser(userId)); // suspends until resolved
  return <h1>{user.name}</h1>;
}

<Suspense fallback={<Skeleton />}>
  <UserProfile userId={1} />
</Suspense>
```

■ Lifecycle & Reconciliation

Q24. What are React lifecycle methods? (Class components)

Lifecycle methods run at specific points: Mounting (constructor, render, componentDidMount), Updating (shouldComponentUpdate, componentDidUpdate), Unmounting (componentWillUnmount).

```
class Timer extends React.Component {
  componentDidMount() { this.id = setInterval(this.tick, 1000); }
  componentWillUnmount() { clearInterval(this.id); }
  tick = () => this.setState(s => ({ time: s.time + 1 }));
  render() { return <p>{this.state.time}s</p>; }
}
```

Q25. How does React's reconciliation algorithm work?

React's 'diffing' algorithm: (1) Two nodes of different types → rebuild the whole subtree. (2) Same type → update attributes. (3) Lists → uses keys to match nodes. This makes updates $O(n)$ instead of $O(n^3)$.

■ *Never change a component's key unnecessarily — it forces a full remount.*

Q26. What is the Fiber architecture?

React Fiber (v16) is a reimplementation of the reconciler enabling incremental rendering: breaking rendering work into chunks, pausing and resuming, prioritizing updates. This enables Concurrent Mode, Suspense, and Transitions.

■ Routing & Forms

Q27. How does React Router work? Explain key components.

React Router enables client-side routing. Key parts: BrowserRouter (history), Routes/Route (matching), Link (navigation), useNavigate / useParams (hooks).

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Link to="/about">About</Link>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/user/:id" element={<User />} />
      </Routes>
    </BrowserRouter>
  );
}
```


Q28. Controlled vs Uncontrolled Components in Forms?

Controlled: form data is driven by React state (single source of truth). Uncontrolled: form data is managed by the DOM; accessed via refs.

```
// Controlled
const [val, setVal] = useState('');
<input value={val} onChange={e => setVal(e.target.value)} />

// Uncontrolled
const ref = useRef();
<input ref={ref} defaultValue="hello" />
// Access: ref.current.value
```

■ Testing & TypeScript

Q29. How do you test React components?

React Testing Library (RTL) is the standard. It tests from the user's perspective. Common: render, screen queries (getByRole, getByText), userEvent, waitFor.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

test('counter increments', async () => {
  render(<Counter />);
  await userEvent.click(screen.getByText('+1'));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

Q30. How do you type React components with TypeScript?

Use React.FC or explicit prop interfaces. Type useState, useRef, and event handlers explicitly.

```
interface ButtonProps {
  label: string;
  onClick: () => void;
  disabled?: boolean;
}

const Button: React.FC<ButtonProps> = ({ label, onClick, disabled }) => (
  <button onClick={onClick} disabled={disabled}>{label}</button>
);

const [user, setUser] = useState<User | null>(null);
const ref = useRef<HTMLInputElement>(null);
```

Best of luck with your React interview! Practice coding, not just reading.